# Consultas SQL avançadas

**Tópico: Junções, agregações, agrupamentos e subconsultas**

- Consultas avançadas combinam tabelas e resumem dados para responder perguntas que uma fonte isolada não resolve.
- Praticaremos JOIN, agregações, GROUP BY, HAVING e subconsultas em um pequeno banco de vendas.
- Os exemplos serão executados no DBeaver com SQLite, relacionando cada cláusula ao resultado obtido.



# Objetivos de aprendizagem

**Tópico: O que deverá ser realizado ao final da aula**

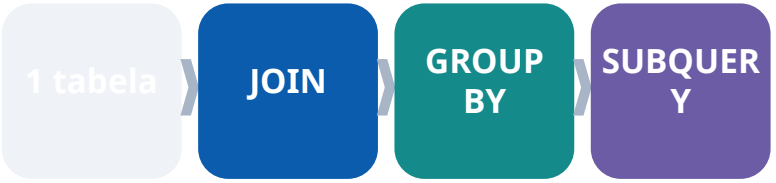
- Relacionar tabelas com INNER JOIN e LEFT JOIN, identificando corretamente as colunas usadas na condição ON.
- Aplicar COUNT, SUM, AVG, MIN e MAX e criar agrupamentos com GROUP BY e HAVING.
- Construir subconsultas com comparações, IN e EXISTS e interpretar seu papel dentro da consulta externa.

1. **Relacionar**
2. **Resumir**
3. **Agrupar**
4. **Aninhar**

# Da consulta básica à consulta avançada

**Tópico: O que muda a partir de agora**

- Consultas básicas selecionam, filtram e ordenam dados; consultas avançadas acrescentam relacionamentos e cálculos sobre conjuntos de linhas.
- JOIN reúne tabelas; agregações resumem valores; GROUP BY forma grupos; subconsultas produzem resultados intermediários.
- Esses recursos podem aparecer juntos, desde que cada cláusula tenha um objetivo claro na resposta.

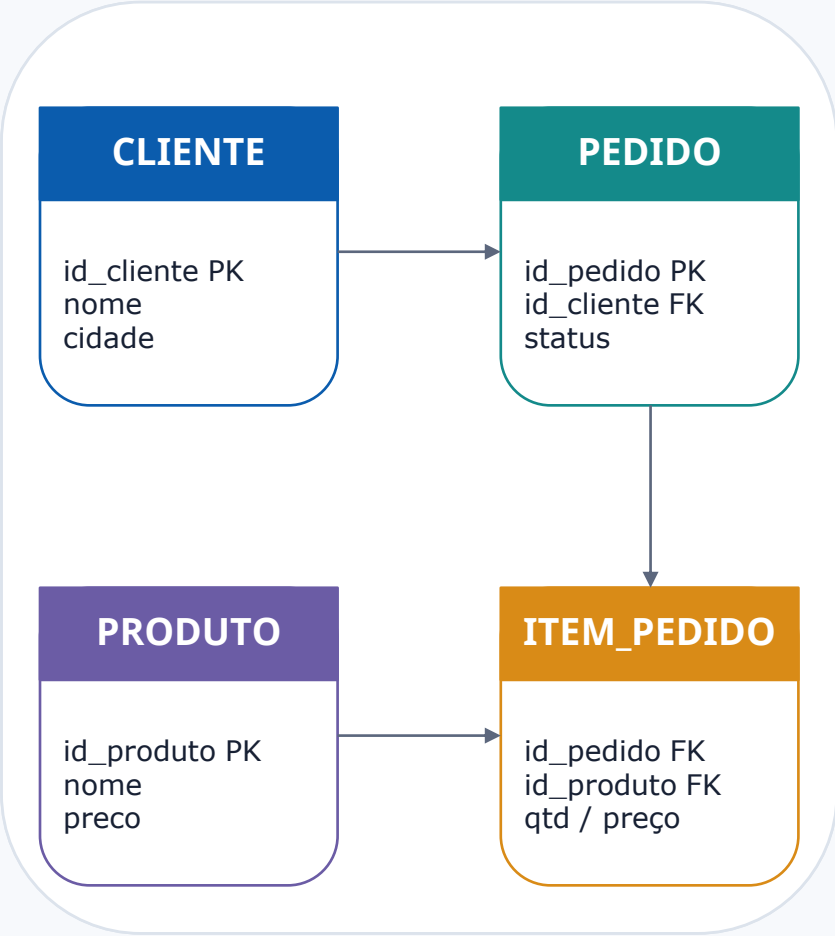



**A complexidade cresce por composição de ideias simples.**

# Cenário didático: banco de vendas

## Tópico: Quatro tabelas relacionadas

- O exemplo usa Cliente, Pedido, Item_Pedido e Produto, ligados por chaves primárias e estrangeiras.
- Um cliente possui pedidos; cada pedido possui itens; cada item referencia um produto e registra quantidade e preço.
- O modelo permite demonstrar junções, totais, médias, agrupamentos e subconsultas sem exigir uma base muito grande.



# Dados usados nos exemplos

**Tópico: Conhecer a base ajuda a prever resultados**

- Ana possui dois pedidos; Bruno, Carla e Diego possuem um; Elisa está cadastrada, mas ainda não comprou.
- Os produtos possuem categorias e preços variados, permitindo calcular totais, médias, mínimos e máximos.
- Conhecer os dados antes da consulta ajuda a prever quantas linhas ou grupos deverão aparecer no resultado.

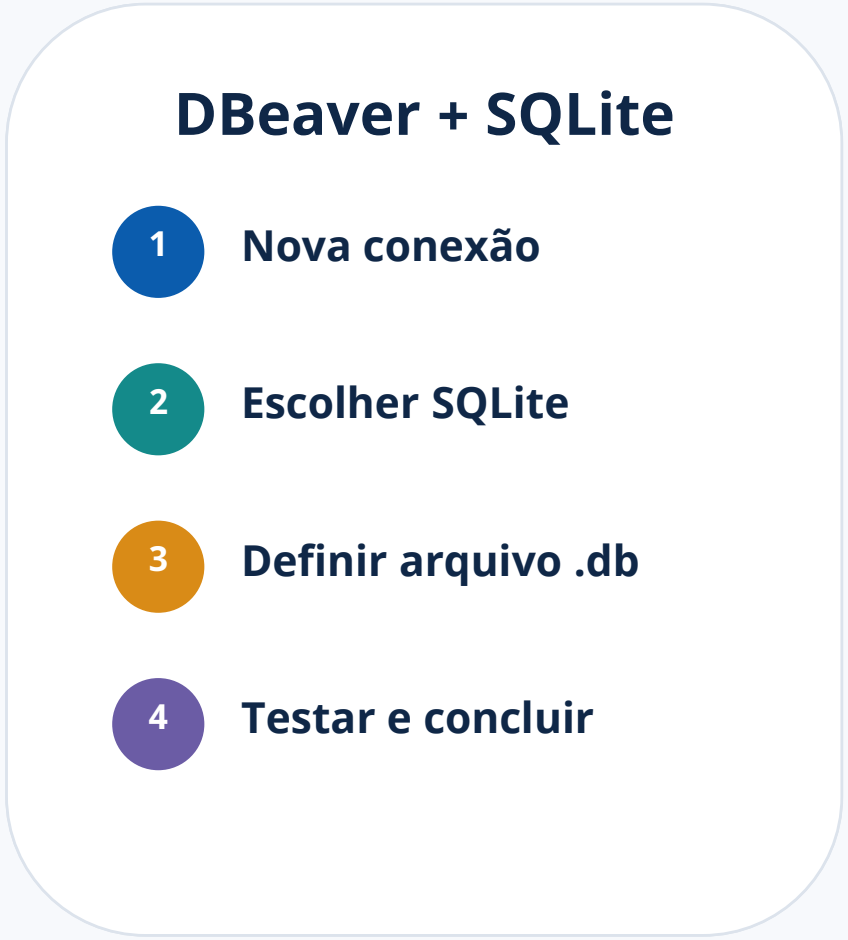
**Clientes e pedidos**

| cliente | pedidos |
|---|---|
| Ana | 101, 102 |
| Bruno | 103 |
| Carla | 104 |
| Diego | 105 |
| Elisa | — |

# DBeaver com SQLite: preparar a conexão

**Tópico: Passo a passo para a prática**

- No DBeaver Community, crie uma nova conexão SQLite e indique um arquivo local, como aula_sql_avancado.db.
- Abra um editor SQL associado à conexão e execute PRAGMA foreign_keys = ON para ativar integridade referencial.
- Como SQLite é incorporado, a prática não exige configurar servidor, usuário ou porta de rede.




DBeaver — conexão SQLite

# DBeaver: executar e conferir consultas

**Tópico: Editor SQL e painel de resultados**

- No editor SQL, escreva uma instrução e pressione Ctrl+Enter para executar a consulta sob o cursor ou seleção.
- O resultado aparece no painel inferior, facilitando conferir colunas, linhas e valores calculados.
- Execute SELECT sqlite_version(); antes da prática para conhecer a versão do SQLite utilizada pelo driver.

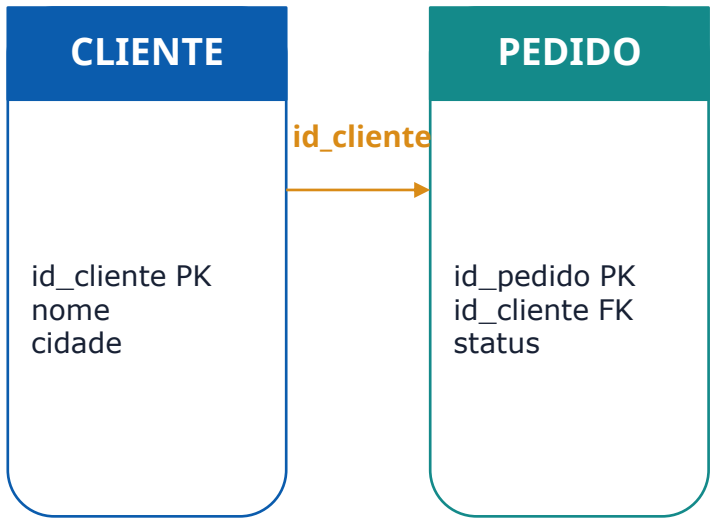



DBeaver — execução SQL

# O que é uma junção?

**Tópico: JOIN combina linhas relacionadas**

- JOIN combina colunas de duas ou mais fontes usando uma condição que determina quais linhas estão relacionadas.
- Pedido.id_cliente referencia Cliente.id_cliente; essa igualdade associa cada pedido ao cliente correspondente.
- O tipo de JOIN define o tratamento das linhas que não encontram correspondência na outra tabela.




**Relacionamento por chave**

# INNER JOIN

## Tópico: Somente linhas com correspondência

- INNER JOIN retorna somente combinações em que a condição ON encontra correspondência entre as tabelas participantes.
- Ao unir Pedido e Cliente, cada pedido recebe o nome do cliente relacionado pelo id_cliente.
- Clientes sem pedidos não aparecem, pois a resposta exige registros existentes nos dois lados da relação.

**Consulta → resultado**

```
SELECT p.id_pedido, c.nome
FROM pedido p
JOIN cliente c
 ON c.id_cliente=p.id_cliente;
```

| pedido | cliente |
|---|---|
| 101 | Ana |
| 102 | Ana |
| 103 | Bruno |
| 104 | Carla |
| 105 | Diego |

SQLite — documentação de SELECT, JOIN e agregações

# INNER JOIN com três tabelas

**Tópico: Seguindo o caminho dos relacionamentos**

- Para descobrir os produtos de cada pedido, percorremos Pedido → Item_Pedido → Produto com duas condições de junção.
- Cada JOIN acrescenta uma nova fonte e uma regra ON que conecta seus registros ao conjunto atual.
- Um pedido com vários itens aparece em várias linhas, comportamento importante para futuras contagens e somas.

**Percorrendo três tabelas**

```
SELECT p.id_pedido, pr.nome,
       i.quantidade
FROM pedido p
JOIN item_pedido i
  ON i.id_pedido=p.id_pedido
JOIN produto pr
  ON pr.id_produto=i.id_produto;
```

SQLite — documentação de SELECT, JOIN e agregações

# Aliases e colunas qualificadas

**Tópico: Consultas legíveis e sem ambiguidade**

- Aliases como c para Cliente e p para Pedido deixam consultas com várias tabelas mais curtas e legíveis.
- Quando nomes de colunas se repetem, c.id_cliente ou p.id_cliente identifica explicitamente a origem do dado.
- Use aliases consistentes em SELECT, ON, WHERE, GROUP BY e ORDER BY para reduzir ambiguidades.

**Aliases de tabela e coluna**

```
SELECT p.id_pedido,
       c.nome AS cliente
FROM pedido AS p
JOIN cliente AS c
  ON c.id_cliente = p.id_cliente;
```

# LEFT JOIN

**Tópico: Preservando todas as linhas da esquerda**

- LEFT JOIN preserva todas as linhas da tabela esquerda e acrescenta correspondências encontradas na tabela direita.
- Se não houver correspondência, as colunas do lado direito aparecem como NULL no resultado.
- Cliente LEFT JOIN Pedido mantém Elisa, permitindo identificar facilmente clientes que ainda não realizaram pedidos.

**Consulta → resultado**

```
SELECT c.nome, p.id_pedido
FROM cliente c
LEFT JOIN pedido p
 ON p.id_cliente=c.id_cliente;
```

| cliente | pedido |
|---|---|
| Ana | 101 |
| Ana | 102 |
| Bruno | 103 |
| Diego | 105 |
| Elisa | NULL |

SQLite — documentação de SELECT, JOIN e agregações

# RIGHT e FULL OUTER JOIN no SQLite

**Tópico: Compatibilidade e verificação de versão**

- Desde o SQLite 3.39.0, RIGHT JOIN e FULL OUTER JOIN também são reconhecidos pelo mecanismo.
- RIGHT preserva o lado direito; FULL preserva ambos os lados e completa ausências com NULL.
- Confirme a versão com SELECT sqlite_version(); antes de depender desses recursos em um ambiente específico.

**Verifique a versão antes**

```
SELECT sqlite_version();

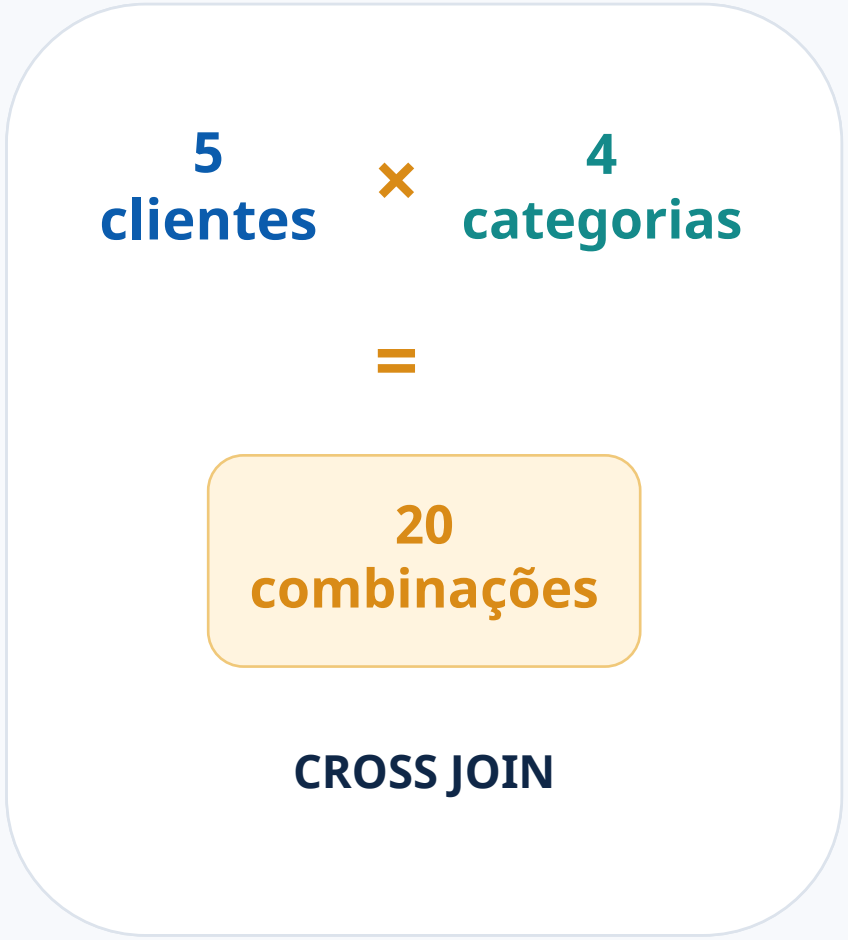
-- SQLite >= 3.39
... RIGHT JOIN ...
... FULL OUTER JOIN ...
```

SQLite 3.39 — RIGHT e FULL OUTER JOIN

# CROSS JOIN e produto cartesiano

**Tópico: Cada linha de um lado com cada linha do outro**

- CROSS JOIN combina cada linha de uma fonte com todas as linhas da outra, sem condição ON.
- Cinco linhas combinadas com quatro linhas produzem vinte resultados antes de qualquer filtro posterior.
- Use esse recurso apenas quando a combinação total for intencional; um JOIN incompleto pode multiplicar dados acidentalmente.




SQLite — documentação de SELECT, JOIN e agregações

# Como escolher o JOIN

**Tópico: Pergunte quais linhas precisam ser preservadas**

- Use INNER JOIN quando a resposta exigir correspondência existente nos dois lados do relacionamento.
- Use LEFT JOIN quando todas as linhas da tabela principal precisarem permanecer, mesmo sem registro relacionado.
- A escolha deve partir da pergunta de negócio e das linhas que precisam ser preservadas no resultado.

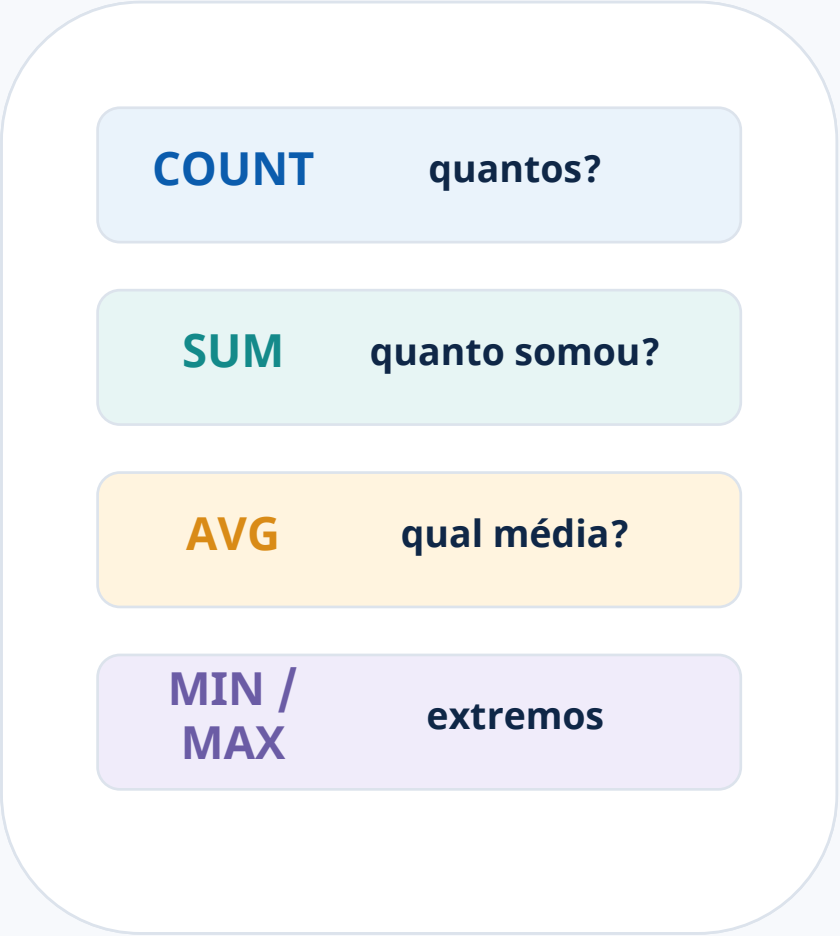



SQLite — documentação de SELECT, JOIN e agregações

# Funções de agregação

**Tópico: Resumindo várias linhas em um valor**

- Funções de agregação resumem várias linhas em medidas como quantidade, soma, média, menor ou maior valor.
- COUNT, SUM, AVG, MIN e MAX são funções fundamentais para relatórios e análises sobre dados relacionais.
- Sem GROUP BY resumem o conjunto inteiro; com GROUP BY são calculadas separadamente para cada grupo.




SQLite — funções de agregação

# COUNT: contando linhas e valores

**Tópico: COUNT(*) não é igual a COUNT(coluna)**

- COUNT(*) conta todas as linhas do conjunto ou grupo, independentemente de uma coluna específica conter NULL.
- COUNT(coluna) conta somente valores não nulos; COUNT(DISTINCT coluna) também elimina repetições antes da contagem.
- Escolha a forma de COUNT conforme a entidade que a pergunta realmente deseja medir.

**Consulta → resultado**

```
SELECT COUNT(*) AS linhas,
 COUNT(DISTINCT id_cliente) AS clientes
FROM pedido;
```

| linhas | clientes |
|---|---|
| 5 | 4 |

SQLite — funções de agregação

# SUM e AVG

**Tópico: Totais e médias calculadas**

- SUM calcula totais e pode receber expressões, como quantidade multiplicada pelo preço unitário de cada item.
- AVG calcula a média dos valores não nulos e retorna uma medida única para o conjunto analisado.
- ROUND pode ser aplicado ao resultado quando a apresentação exigir quantidade controlada de casas decimais.

**Consulta → resultado**

```
SELECT
SUM(quantidade*preco_unitario)
total,
 AVG(preco_unitario) AS media
FROM item_pedido;
```

| total | média |
|---|---|
| 6970,00 | 736,25 |

SQLite — funções de agregação

# MIN e MAX

**Tópico: Encontrando extremos do conjunto**

- MIN retorna o menor valor não nulo do conjunto e MAX retorna o maior valor encontrado.
- Em Produto, essas funções permitem localizar rapidamente os extremos de preço sem ordenar manualmente a tabela.
- Quando precisamos das demais colunas da linha extrema, outra estratégia, como subconsulta, pode ser necessária.

**Consulta → resultado**

```
SELECT MIN(preco) AS menor,
       MAX(preco) AS maior
FROM produto;
```

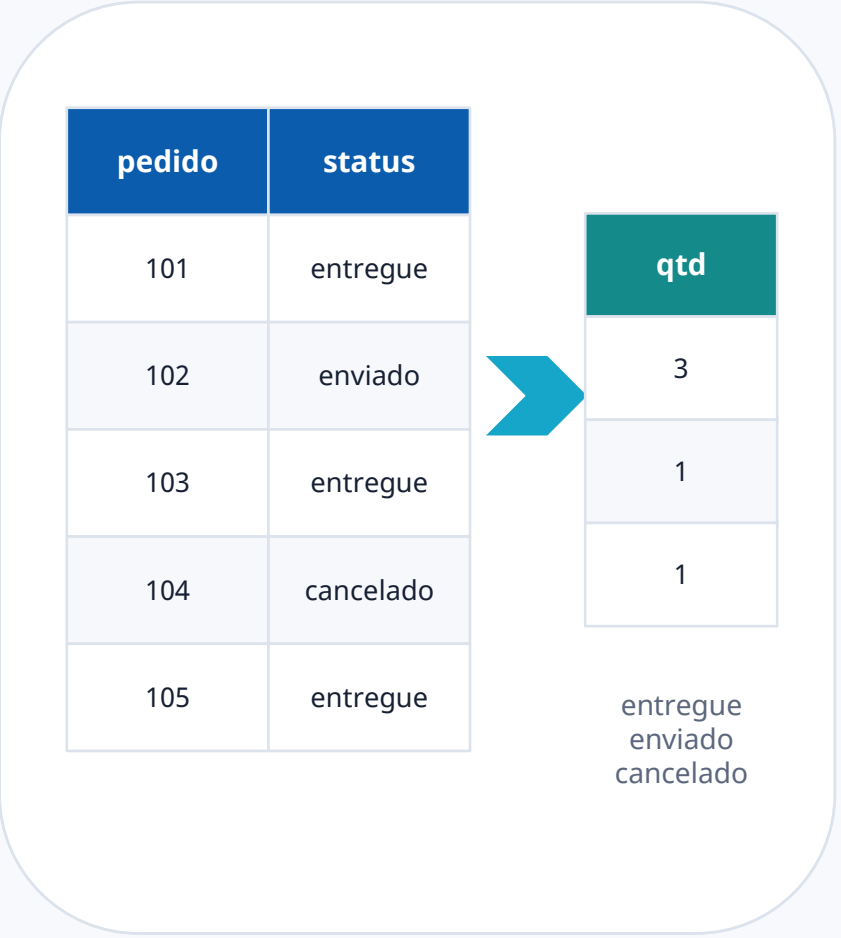
| menor | maior |
|---|---|
| 80,00 | 3500,00 |

SQLite — funções de agregação

# GROUP BY

**Tópico: Transformando linhas em grupos**

- GROUP BY reúne linhas que possuem o mesmo valor, ou a mesma combinação de valores, nas expressões escolhidas.
- Cada grupo contribui com uma linha de resultado e recebe seus próprios cálculos agregados.
- Agrupar Pedido por status permite contar entregues, enviados e cancelados em uma única consulta.




SQLite — documentação de SELECT, JOIN e agregações

# GROUP BY com mais de uma coluna

**Tópico: Grupos definidos por combinações**

- Com várias colunas no GROUP BY, cada grupo é definido pela combinação dos valores dessas colunas.
- Agrupar por id_cliente e status separa cada cliente conforme as diferentes situações de seus pedidos.
- Como prática portável, colunas exibidas fora de agregações devem representar claramente a identidade do grupo.

**Grupo = combinação de valores**

```
SELECT p.id_cliente, p.status,
       COUNT(*) AS qtd
FROM pedido p
GROUP BY p.id_cliente, p.status
ORDER BY p.id_cliente;
```

SQLite — documentação de SELECT, JOIN e agregações

# WHERE ou HAVING?

**Tópico: Filtrar linhas antes ou grupos depois**

- WHERE filtra linhas antes do agrupamento; HAVING filtra grupos depois que as agregações foram calculadas.
- HAVING pode usar condições como COUNT(*) >= 2 para manter apenas grupos que atendem ao limite.
- Uma mesma consulta pode usar WHERE e HAVING quando existem critérios individuais e critérios agregados.

**Consulta → resultado**

```
SELECT c.nome, COUNT(*) AS qtd
FROM cliente c
JOIN pedido p ON
p.id_cliente=c.id_cliente
GROUP BY c.id_cliente,c.nome
HAVING COUNT(*) >= 2;
```

| cliente | qtd |
|---|---|
| Ana | 2 |

SQLite — documentação de SELECT, JOIN e agregações

# Agregação depois de JOIN

**Tópico: Somando informações de várias tabelas**

- Consultas analíticas frequentemente executam JOIN primeiro e depois agregam o conjunto combinado por cliente ou categoria.
- Para calcular compras por cliente, some quantidade × preço_unitario depois de relacionar Cliente, Pedido e Item_Pedido.
- Filtrar pedidos cancelados antes do GROUP BY evita que valores inválidos participem dos totais finais.

**Consulta → resultado**

```
SELECT c.nome,

SUM(i.quantidade*i.preco_unitari
o) AS total
FROM cliente c JOIN pedido p ...
WHERE p.status <> 'cancelado'
GROUP BY c.id_cliente, c.nome;
```

| cliente | total |
|---|---|
| Ana | 4560 |
| Diego | 1780 |
| Bruno | 550 |

SQLite — documentação de SELECT, JOIN e agregações

# COUNT(DISTINCT ...) após JOIN

**Tópico: Evitar contagens infladas pelos itens**

- Após unir Pedido a Item_Pedido, um mesmo pedido aparece várias vezes quando possui mais de um item.
- COUNT(*) pode então contar itens, embora a pergunta desejasse contar pedidos; COUNT(DISTINCT p.id_pedido) corrige essa diferença.
- Visualize algumas linhas do JOIN antes de agregar para entender exatamente o que será contado.

**Contar a entidade correta**

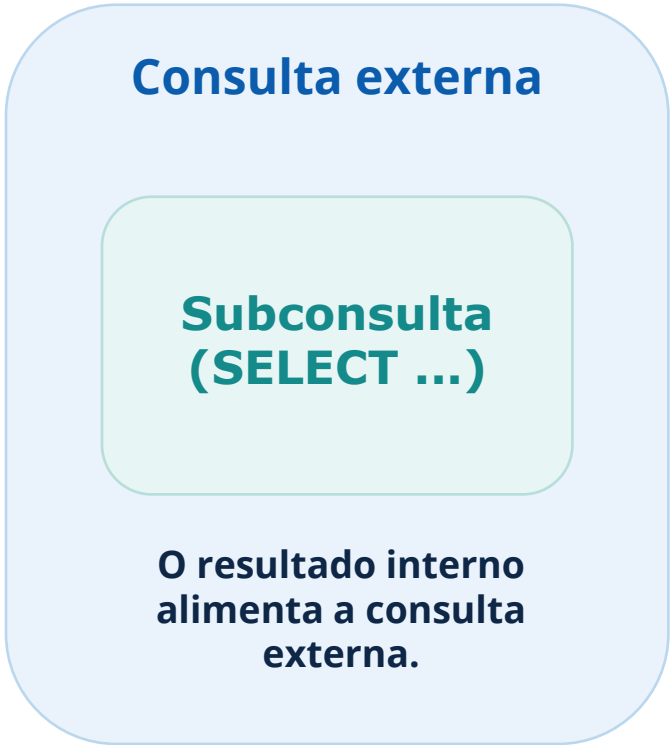
```
SELECT COUNT(*) AS
       COUNT(DISTINCT
p.id_pedido)
        AS pedidos_reais
FROM pedido p
JOIN item_pedido i
  ON i.id_pedido=p.id_pedido;
```

SQLite — documentação de SELECT, JOIN e agregações

# O que é uma subconsulta?

**Tópico: Uma consulta dentro de outra consulta**

- Subconsulta é um SELECT usado dentro de outra instrução para produzir um valor, lista ou conjunto intermediário.
- A consulta interna aparece entre parênteses e seu resultado participa da condição ou da fonte externa.
- Execute primeiro a parte interna isoladamente para confirmar o resultado antes de montar a consulta completa.




SQLite — documentação de SELECT, JOIN e agregações

# Subconsulta escalar

**Tópico: A consulta interna retorna um único valor**

- Uma subconsulta escalar retorna um único valor que pode participar de uma comparação na consulta externa.
- Para encontrar produtos acima da média, calcule AVG(preco) internamente e compare cada preço com esse resultado.
- Na base de exemplo, apenas o Notebook possui preço superior à média geral dos produtos.

**Consulta → resultado**

```
SELECT nome, preco
FROM produto
WHERE preco > (
 SELECT AVG(preco) FROM
produto
);
```

| produto | preço |
|---|---|
| Notebook | 3500 |

SQLite — documentação de SELECT, JOIN e agregações

# Subconsulta com IN

**Tópico: A consulta interna retorna uma lista**

- IN pode comparar um valor com a lista produzida por uma subconsulta, sem escrever identificadores manualmente.
- Podemos selecionar clientes cujo id_cliente aparece entre pedidos com status entregue na consulta interna.
- Esse padrão é útil quando a pergunta pode ser formulada como “está entre os valores encontrados?”.

**Consulta → resultado**

```
SELECT nome FROM cliente
WHERE id_cliente IN (
 SELECT id_cliente FROM pedido
 WHERE status='entregue'
);
```

| cliente |
| --- |
| Ana |
| Bruno |
| Diego |

SQLite — documentação de SELECT, JOIN e agregações

# EXISTS e subconsulta correlacionada

**Tópico: Verificando se existe pelo menos uma linha relacionada**

- EXISTS é verdadeiro quando a subconsulta encontra pelo menos uma linha e responde bem a perguntas de existência.
- Na forma correlacionada, a consulta interna referencia a linha atual da consulta externa, como o id_cliente.
- NOT EXISTS inverte a lógica e permite localizar clientes que ainda não possuem pedidos relacionados.

**Subconsulta correlacionada**

```
SELECT c.nome
FROM cliente c
WHERE EXISTS (
 SELECT 1
 FROM pedido p
 WHERE
p.id_cliente=c.id_cliente
);
```

SQLite — documentação de SELECT, JOIN e agregações

# Subconsulta no FROM

**Tópico: Resultado intermediário tratado como tabela**

- Uma subconsulta no FROM produz um conjunto tabular intermediário que recebe alias e funciona como outra fonte.
- Esse recurso separa uma transformação interna da agregação externa, facilitando a leitura de consultas mais complexas.
- No exemplo, a parte interna classifica preços e a parte externa conta produtos em cada faixa.

**Subconsulta como tabela**

```
SELECT faixa, COUNT(*) AS
produtos
FROM (
 SELECT CASE WHEN preco >=
500
   THEN 'alto' ELSE 'baixo' END AS
faixa
 FROM produto
) AS x
GROUP BY faixa;
```

SQLite — documentação de SELECT, JOIN e agregações

# Ordem lógica da consulta avançada

**Tópico: Um modelo mental para ler SQL complexo**

- Para raciocinar, comece por FROM e JOIN; depois aplique WHERE, GROUP BY e HAVING.
- SELECT calcula as colunas finais e ORDER BY organiza a apresentação do resultado produzido.
- O otimizador pode executar fisicamente de outra maneira, mas precisa preservar a semântica definida pela consulta.




**Ordem lógica para raciocinar**

SQLite — documentação de SELECT, JOIN e agregações

# Prática guiada no DBeaver

**Tópico: Construindo a consulta por etapas**

- Primeiro una Pedido e Cliente e confirme se cada pedido recebeu o nome correto do cliente.
- Depois acrescente Item_Pedido, calcule valores por linha e somente então aplique SUM e GROUP BY.
- Finalize excluindo pedidos cancelados e ordenando os totais, comparando cada etapa no painel do DBeaver.



# Erros comuns em SQL avançado

**Tópico: Consultas podem executar e ainda estar erradas**

- Uma condição ON ausente ou incorreta pode multiplicar linhas e aumentar artificialmente somas e contagens.
- COUNT(*) após relacionamento um-para-muitos pode medir itens quando a intenção era contar pedidos distintos.
- Teste subconsultas isoladamente e diferencie filtros de linha em WHERE dos filtros agregados em HAVING.

| | |
|---|---|
| ! **JOIN sem ON** | linhas multiplicadas |
| ! **COUNT(*)** | entidade errada |
| ! **COUNT no WHERE** | use HAVING |
| ! **subquery não testada** | resultado inesperado |

# Desafio integrado

**Tópico: JOIN + GROUP BY + HAVING + ORDER BY**

- O desafio pede clientes com compras não canceladas de pelo menos 1.000, exibindo nome e total comprado.
- Una as tabelas necessárias, filtre cancelados, agrupe por cliente e some quantidade × preço_unitario.
- Use HAVING para aplicar o limite ao total agregado e ORDER BY para mostrar os maiores valores primeiro.

**Uma solução possível**

```
SELECT c.nome,

ROUND(SUM(i.quantidade*i.preco_unit
ario),2) AS total
FROM cliente c
JOIN pedido p ON
p.id_cliente=c.id_cliente
JOIN item_pedido i ON
i.id_pedido=p.id_pedido
WHERE p.status <> 'cancelado'
GROUP BY c.id_cliente,c.nome
HAVING
SUM(i.quantidade*i.preco_unitario)
>= 1000
ORDER BY total DESC;
```

# Avaliação rápida e referências

**Tópico: Verificação de aprendizagem e fontes**

- Avaliação: explique INNER versus LEFT JOIN e diferencie o papel de WHERE do papel de HAVING.
- Escreva uma contagem de pedidos por status e uma subconsulta para produtos com preço acima da média.
- Referências: documentação oficial do SQLite sobre SELECT e agregações e documentação do DBeaver sobre SQLite e execução SQL.

## Fontes oficiais

**SQLite — SELECT / JOIN**

**SQLite — Agregações**

**DBeaver — SQLite**

**DBeaver — Execução SQL**

SQLite • DBeaver — documentação oficial